Atlas Fulfillment and Atlas Checkout accelerator for Meridian OMS

> **About this sample**
>
> **Document type:** Integration reference
> **Audience:** Enterprise architects and OMS developers
> **Purpose:** Describes the architecture, modules, and deployment of a pre-built integration between an enterprise order management system (OMS) and a commerce and fulfillment platform.
> **Note:** Company, product, and platform names are fictional. The technical content is representative of the published version.

You can use the Atlas Fulfillment and Atlas Checkout accelerator for Meridian OMS to integrate Atlas Fulfillment and Atlas Checkout with your Meridian order management system (OMS). The following sections describe the technical requirements, solution architecture, implementation modules, and deployment considerations for the accelerator.

Accelerator benefits

The Atlas Fulfillment and Atlas Checkout accelerator for Meridian OMS reduces integration time and technical complexity. You can use the accelerator to implement Atlas commerce capabilities without extensive modifications to your existing inventory management, order processing, fulfillment tracking, and returns handling systems.

The accelerator works alongside your current OMS configurations with minimal system changes. This approach lets development teams focus on merchant-specific customizations instead of building core integration components.

Solution overview

The Atlas Fulfillment and Atlas Checkout accelerator for Meridian OMS processes data asynchronously without disrupting your existing Meridian OMS operations. The accelerator manages data transformations and maintains state consistency between Atlas services and your OMS, adding Atlas fulfillment capabilities while preserving existing workflows.

Use cases

The following table describes several use cases for the Atlas Fulfillment and Atlas Checkout accelerator for Meridian OMS:

Use case	From	To	Description
Fulfill with Atlas	OMS	Atlas	Create and manage Atlas Fulfillment and Atlas Checkout orders from Meridian OMS.
Atlas fulfillment updates	Atlas	OMS	Update your Meridian OMS with delivery cancellations, delivery failures, shipping, and tracking info from Atlas.
Returns through Atlas	Atlas	OMS	Handle returns initiated from Atlas, create return orders in Meridian OMS, and sync return delivery status.
Returns outside Atlas	OMS	Atlas	Sync merchant-initiated returns with Atlas and maintain return visibility.
Atlas refund requests	Atlas	OMS	Process refund requests initiated from Atlas due to delivery cancellations, delivery failures, and returns.
Synchronize issued refunds	OMS	Atlas	Send refund updates from OMS to Atlas to ensure synchronized status.
Merchant order cancellation	OMS	Atlas	Request order cancellation through various channels.
Synchronize inventory	Atlas	OMS	Sync inventory in real time and batch from Atlas to Meridian OMS to maintain accurate stock levels.

Design principles and technical architecture

Integration principles

The Atlas Fulfillment and Atlas Checkout accelerator for Meridian OMS uses principles such as keeping client systems authoritative, maintaining system boundaries, and using asynchronous, event-driven communication.

Architecture layers

The Atlas Fulfillment and Atlas Checkout accelerator for Meridian OMS uses three distinct architectural layers:

- Atlas Connector layer: Handles technical connectivity and data mapping between Atlas and OMS formats.
- OMS Integration Services layer: Handles business logic and orchestration.
- OMS Event Handler layer: Provides validation and routing logic. You deploy the OMS Event Handler layer on your OMS platform to identify and direct Atlas orders to the accelerator's OMS Integration Services. This layer serves as a reference implementation that you can configure or extend based on your requirements.

Event processing architecture

The architecture of the accelerator uses asynchronous event processing to reduce system coupling and increase scalability.

- Atlas to Meridian OMS: Webhooks send Atlas events to the Atlas Connector, which validates and transforms data before routing to Java Message Service (JMS) queues. The OMS Integration Services layer processes these events asynchronously, maintains OMS state, and coordinates Atlas responses. Meridian OMS uses PKCS#12 certificates and basic authentication for security. The accelerator processes JSON-based Atlas events and communicates with Meridian OMS using the native XML format that its APIs require.
- Meridian OMS to Atlas: The accelerator validates Meridian OMS transaction events for Atlas orders and routes them to JMS queues for processing. The OMS Integration Services layer processes these events asynchronously, maintains Atlas state, and coordinates OMS responses. Access to Atlas services requires OAuth-based authentication. The accelerator communicates with Atlas services using Atlas Checkout APIs and Atlas Seller APIs.

Meridian OMS platform integration

The accelerator design aligns with Meridian OMS order lifecycle transactions, events, and processing pipeline. This design principle supports integration with your existing OMS implementation and reduces customization requirements. The architecture uses established Meridian OMS mechanisms for order processing, fulfillment, and inventory management.

Accelerator configuration

The Atlas Fulfillment and Atlas Checkout accelerator for Meridian OMS lets you configure properties to change the behavior of the accelerator based on your business needs and account setup. You can modify these properties using API calls or manually modify them with the Meridian OMS application manager.

Merchant-specific customizations and configurations

Minimize impact to your Meridian OMS environment

To minimize impact to your Meridian OMS environment, the Atlas Fulfillment and Atlas Checkout accelerator for Meridian OMS does the following:

- Excludes configurations like hub rules, user exits, and event actions from the installation package to preserve your existing Meridian OMS configurations.
- Externalizes common, potentially overlapping configurations like transaction and status codes.
- Provides sample pipelines for order, shipment, and return processing to avoid conflicts with your existing pipelines.
- Processes most accelerator operations in a separate transaction boundary from core OMS transactions to minimize performance impact.
- Prefixes all Atlas configurations with `Atl` to avoid conflicts with your existing setup.

Required configurations and customizations

To use the Atlas Fulfillment and Atlas Checkout accelerator for Meridian OMS, you need to apply the following common configurations and customizations. Your specific Meridian OMS implementation might require additional customizations.

- Configurations: Review and assess configurations like conflicting status codes and ship node values for applicability.
- Pipelines: The accelerator provides sample pipelines for order, shipment, and return processing. You might need to modify these to include common transactions such as sales and returns posting.
- Servers: The accelerator includes several server configurations, but you might prefer to consolidate infrastructure usage with your existing servers for common processes like creating orders and scheduling and releasing orders. This requires manual changes to the server configurations after deployment.
- OMS Event Handler: Implement the OMS Event Handler. We provide this as a reference implementation but do not include it in the accelerator deployment package.
- Code: You can make changes to the accelerator source code as necessary. For example, you can update the source code to add support for segmentation when updating inventory.

Implementation modules

The Atlas Fulfillment and Atlas Checkout accelerator for Meridian OMS uses several modules to integrate Atlas services with Meridian OMS. These modules handle order management, fulfillment, returns processing, refund management, cancellations, and inventory synchronization. Each module uses specific APIs and event triggers to maintain data consistency between systems.

Fulfill with Atlas

The Fulfill with Atlas module supports both storefront-initiated and OMS-initiated orders. For OMS-initiated orders, the module handles creation and routing to Atlas, managing the fulfillment initialization process. For storefront-initiated orders, the accelerator initiates fulfillment.

The following table lists fulfillment APIs and events:

Component type	Name	Purpose
Atlas APIs	`mutation: createOrder`	Creates new order in Atlas.
	`mutation: updateOrder`	Updates order status.
Atlas events	N/A	-
OMS APIs	`getOrderList`	Get complete order details.
	`changeOrder`	Update order with Atlas `OrderId`; cancel OMS order line if failed to create Atlas order.
OMS events	`ORDER_CREATE.0001.ON_SUCCESS`	Triggers on creation of OMS order, used to create Atlas order or initiate Atlas fulfillment.
	`RELEASE.0001.ON_SUCCESS`	Triggers on release of OMS order, used to initialize Atlas fulfillment.

Atlas fulfillment updates

The fulfillment status update component provides real-time visibility through event-based status updates. It processes various fulfillment events including in-transit notifications, delivery confirmations, package milestones, cancellations, and delivery failures. After Atlas starts fulfilling an order, this module automatically updates OMS with package delivery milestones such as current package location, delivery attempts, and package status.

After the accelerator creates a shipment in OMS, your OMS is expected to create a shipment invoice and complete the payment processing.

The following table lists shipment and delivery APIs and events:

Component type	Name	Purpose
Atlas APIs	`query: order`	Retrieves delivery/package details.
Atlas events	`PACKAGE_DELIVERY_IN_TRANSIT`	Indicates shipment started.
	`PACKAGE_DELIVERED`	Confirms delivery completion.
OMS APIs	`getShipmentContainerList`	Get shipment details for `AtlDeliveryId`.
	`getOrderLineList`	Get order line details for Atlas item alias.
	`createShipment`	Creates shipment record.
	`changeShipmentStatus`	Updates shipment status.

The following table lists package delivery milestone APIs and events:

Component type	Name	Purpose
Atlas APIs	`query: Order`	Retrieves tracking details.
Atlas events	`PACKAGE_TRACKER_MILESTONE_CHANGED`	Updates tracking milestones.
OMS APIs	`getShipmentContainerList`	Get shipment details for `TrackingNo`.
	`getContainerMilestoneList` (custom)	Get milestones for `AtlDeliveryId` and `TrackingNo`.
	`updateTrackingMilestones` (custom)	Updates tracking information.

The following table lists delivery cancellation and delivery failure APIs and events:

Component type	Name	Purpose
Atlas APIs	`query: order`	Retrieves delivery/package details.
Atlas events	`PACKAGE_DELIVERY_CANCELLED`	Indicates delivery cancellation.
	`PACKAGE_DELIVERY_FAILURE`	Indicates delivery failures.
OMS APIs	`getShipmentContainerList`	Get shipment details for `AtlDeliveryId`.
	`getOrderLineList`	Get order line details for Atlas item alias.
	`changeRelease`	Cancel release.
	`changeOrder`	Cancel for Atlas Checkout or backorder line for Atlas Fulfillment.
	`changeShipmentStatus`	Updates shipment status.

Returns through Atlas

This module processes returns initiated through the Atlas online returns center or Atlas Checkout shopper order details page. It automatically creates return records in OMS and maintains synchronization of return delivery status updates, ensuring consistent return tracking across systems.

The following table lists Atlas return APIs and events:

Component type	Name	Purpose
Atlas APIs	`query: Order`	Retrieves return details.
Atlas events	`RETURN_STARTED`	Indicates return initiation.
	`RETURN_PACKAGE_IN_TRANSIT`	Updates return shipment status.
	`RETURN_PACKAGE_DELIVERED`	Confirms return delivery.
	`RETURN_ITEM_GRADED`	Updates return item inspection status.
	`RETURN_COMPLETED`	Indicates return completion.
OMS APIs	`getOrderList`	Get order details for `AtlReturnId`.
	`getOrderLineList`	Get order line details for `AtlOrderId`.
	`createOrder`	Creates return order in OMS.
	`changeOrderStatus`	Change order status.
	`changeOrder`	Save events.

Returns outside Atlas

This module manages returns initiated outside the Atlas platform, such as through your OMS customer service portal. It ensures Atlas has visibility into all returns, preventing duplicate return authorizations.

The following table lists external return APIs and events:

Component type	Name	Purpose
Atlas APIs	`mutation: updateOrder`	Creates or updates return in Atlas.
OMS APIs	`getOrderList`	Retrieves order details.
	`changeOrder`	Updates return status.
OMS events	`ORDER_CREATE.0003.ON_SUCCESS`	Triggers external return processing.
	`ORDER_CHANGE.0003.ON_SUCCESS`	Triggers return status updates.

Atlas refund requests

This module processes refund request notifications from Atlas. It handles refund requests for various scenarios including successful returns, order cancellations, and delivery failures.

After the accelerator creates a return invoice or a credit memo, your OMS is expected to complete the payment refund processing.

The following table lists refund request APIs and events:

Component type	Name	Purpose
Atlas APIs	`mutation: updateOrder`	Updates refund status in Atlas.
	`query: Order`	Query refund details.
Atlas events	`REFUND_REQUESTED`	Triggers refund processing.
OMS APIs	`getReceiptLineList`	Get receipts for `AtlReturnId` and `AtlRefundId`.
	`receiveOrder`	Mark return line as received.
	`closeReceipt`	Close receipt.
	`createOrderInvoice`	Create return invoice.
	`changeOrderInvoice`	Update invoice with `AtlRefundId`.
	`recordInvoiceCreation`	Creates credit memo invoice.
	`getInvoiceDetails`	Retrieves invoice based on `AtlRefundId`.
	`getOrderLineList`	Retrieve order lines based on item ID alias.

Synchronize issued refunds

This module updates Atlas with refund processing status and details after OMS completes refund payment processing. This ensures Atlas has accurate records of completed refunds and can update customer communications accordingly.

The following table lists credit memo refund APIs and events:

Component type	Name	Purpose
Atlas APIs	`mutation: updateOrder`	Updates refund status in Atlas.
OMS events	`PAYMENT_COLLECTION.ON_INVOICE_COLLECTION`	Triggers refund status updates.

The following table lists Atlas-initiated refund APIs and events:

Component type	Name	Purpose
Atlas APIs	`mutation: updateOrder`	Updates refund status in Atlas.
OMS events	`PAYMENT_COLLECTION.ON_INVOICE.0003_COLLECTION`	Triggers refund status updates.

The following table lists merchant-initiated refund APIs and events:

Component type	Name	Purpose
Atlas APIs	`mutation: updateOrder`	Updates refund status in Atlas.
OMS APIs	`getOrderList`	Retrieve order details.
	`getInvoiceDetailList`	Retrieve invoice details.
	`changeOrderInvoice`	Update invoice with `AtlRefundId`.
OMS events	`PAYMENT_COLLECTION.ON_INVOICE.0003_COLLECTION`	Triggers refund status updates.

Merchant order cancellation

You can initiate cancellation of Atlas Checkout or Atlas Fulfillment orders through various channels like your website or call center. You can cancel orders for reasons such as customer remorse, out-of-stock items, payment issues, or other business scenarios. The accelerator evaluates and forwards valid cancellation requests to Atlas for processing.

The following table lists order cancellation APIs and events:

Component type	Name	Purpose
Atlas APIs	`mutation: cancelOrder`	Request to cancel Atlas order.
OMS events	`CHANGE_ORDER_ON_CANCEL`	Triggers request to cancel Atlas order.

Synchronize inventory

The inventory management module synchronizes inventory between Atlas and your Meridian OMS. This module supports both real-time inventory updates through events and periodic full synchronization to ensure accurate inventory levels across systems. The periodic full synchronization operates through a scheduled agent process that systematically retrieves inventory data from Atlas and updates the OMS.

The following table lists real-time inventory APIs and events:

Component type	Name	Purpose
Atlas APIs	`getInventorySummaries`	Get fulfillable quantity for seller `SKU`.
Atlas events	`INVENTORY_CHANGED`	Triggered on inventory availability change.
OMS APIs	`getAggregatedDemand`	Get sum of all open demand.
	`syncSupply`	Update supply quantity.

The following table lists periodic inventory synchronization APIs:

Component type	Name	Purpose
Atlas APIs	`getInventorySummaries`	Get fulfillable quantity for batch of seller `SKUs`.
	`query: Products`	Get external ID for seller `SKU`.
OMS APIs	`getAggregatedDemand`	Get sum of all open demand.
	`syncSupply`	Update supply quantity.

Installation and deployment overview

Prerequisites

The installation process requires a functioning Meridian OMS environment with a Java Message Service (JMS) MQ platform, core components, code packages, and configurations. The accelerator supports both on-premises and cloud-hosted editions of Meridian OMS.

Deployment

We recommend the following approach to deploy the accelerator:

1. Deploy to a sandbox environment to identify gaps and conflicts.
2. Merge configurations into the management configuration environment.

The implementation spans multiple layers, including application code deployment, JMS queue configuration, and server infrastructure setup. Module-specific configurations require enabling specific primitives, setting module properties, configuring database extensions, and establishing integration points aligned with your business requirements.